

Programación 3

**T.P.I: FoodStore frontend +
backend.**

Alumno: Ovelar Isaias Javier

Fecha:02/07/26

Contenido

Front End	3
Marco Teórico	3
Decisiones Técnicas y Arquitectura	3
Problemas y soluciones.	4
Back End.....	6
Marco Teórico	6
Problemas y soluciones	7
Bibliografía.....	9

Front End

Marco Teórico

El proyecto, denominado “FoodStore” se trata de una aplicación de e-commerce, dedicado a la venta de alimentos y bebidas.

Para el desarrollo del front end, se hizo uso de HTML, TypeScript con Vite, CSS puro (sin Tailwind), utilizando el gestor de paquetes pnpm. Se aplicó la propuesta de catedra en la cual indicaba que se podría presentar front end y backend sin conexión.

HTML: lenguaje de marcado utilizado en la creación de páginas web. Define una estructura básica y un código (denominado código HTML) para la presentación de contenido de una página web, que incluye texto, imágenes, videos, juegos, entre otros elementos.

CSS: lenguaje informático especializado en definir y cohesionar la presentación de un documento escrito en un [lenguaje de marcado](#) como [HTML](#) o [XML](#). Está diseñado principalmente para marcar la separación del contenido del documento y la forma de presentación de este, características tales como las capas o *layouts*, los colores y las fuentes.^[4]

Vite: Vite (palabra francesa para "rápido") es una herramienta de compilación que tiene como objetivo proporcionar una experiencia de desarrollo más rápida y ágil para proyectos web modernos.

TypeScript es un lenguaje de programación fuertemente tipado que se basa en JavaScript, brindándote mejores herramientas a cualquier escala. Añade sintaxis adicional a JavaScript para admitir una integración más fluida con tu editor, lo que permite detectar errores de forma temprana. Se convierte a JavaScript, el cual se ejecuta en cualquier lugar donde este lo haga: en un navegador, en Node.js, Deno, Bun y en tus aplicaciones.

PNPM: Abreviatura de Performant npm (npm de alto rendimiento), es uno de los principales gestores de paquetes de JavaScript. Fue desarrollado originalmente en 2016 por Rico Sta. Cruz para el entorno de ejecución de JavaScript Node.js, y se enfoca en ser una alternativa a npm que optimiza el uso del espacio en el disco duro.

Decisiones Técnicas y Arquitectura

Código base: Se reciclo parte del código del parcial 1

Separación de responsabilidades. El proyecto se estructuró en tres capas claras: *types/* (modelos/contratos de datos), *utils/* (servicios reutilizables) y *pages/* (vistas y su lógica, una carpeta por ruta). Esta organización evita duplicar código y mantiene cada archivo con una única responsabilidad.

Capa de acceso a datos desacoplada. Todas las lecturas se hacen con `fetch()` a archivos locales (`/data/*.json`). Cada vista accede de forma independiente a `localStorage`

Manejo de errores. Al no existir un servidor, el manejo de excepciones se realiza en el punto de consumo: cada fetch se envuelve en try/catch y se validan las respuestas y los campos de los formularios (por ejemplo, credenciales inválidas en el login o stock insuficiente en el carrito), mostrando mensajes al usuario en lugar de dejar que el error se propague

Problemas y soluciones.

Más allá de los problemas normales a la hora de desarrollar, que en su mayoría son errores humanos y no merecen mención, hay una serie de problemas (que recuerde) que se merecen destacar:

Ocultar ciertos botones del rol administrador:

Se asume que el administrador puede tener acceso a la tienda, pero no puede realizar compras; por ende, no puede agregar ítems al carrito, ni disponer de acceso al carrito ni a "Mis Pedidos".

Para solucionar esto se hizo uso de una clase denominada "es-admin", la cual actúa como un "interruptor":

```
.es-admin .product-card__btn,  
.es-admin #btnAgregar,  
.es-admin .nav__link--cart,  
.es-admin .nav__link--pedidos {  
  display: none;  
}
```

Mediante la propiedad `display: none` se ocultan simultáneamente los botones de "Agregar al carrito" (de las tarjetas y del detalle) y los enlaces de navegación al carrito y a "Mis Pedidos".

Esto se realiza desde el guard de las vistas de la tienda (Home y Detalle de producto), que es donde existen dichos elementos. El guard valida el rol del usuario recuperado desde `localStorage` y, solo si es "ADMIN", agrega la clase al body:

```
function initGuard() {  
  const raw = getUser(); // 1. obtiene el  
  usuario de localStorage  
  if (!raw) {  
    navigate("/src/pages/auth/login/login.html");  
    return false;  
  }  
  const user = JSON.parse(raw); // 2. lo convierte a  
  objeto  
  
  if (user.role === "ADMIN" && adminLink)  
    adminLink.style.display = "";
```

```
if (user.role === "ADMIN") document.body.classList.add("es-admin"); // 3. valida y asigna
return true;
}
```

Ejemplos de como se oculta siendo administrador

Food Store

[Inicio](#)
[Administración](#)
[Cerrar Sesión](#)

Categorías

- Todas las categorías
- Hamburguesas
- Pizzas
- Empanadas
- Bebidas
- Papas Fritas

Ordenar por

Todos



Hamburguesas
Hamburguesa Clásica
Carne 150g, lechuga, tomate, cebolla morada y mayonesa

\$4500



Hamburguesas
Hamburguesa Doble
Doble carne 150g c/u, cheddar, panceta y salsa barbacoa

\$6200



Hamburguesas
Hamburguesa Completa
Carne 150g, huevo, jamón, queso, lechuga y tomate

\$5500

Food Store

[Inicio](#)
[Cerrar Sesión](#)



Hamburguesa Clásica

\$4500

Disponible (Stock: 50)

Carne 150g, lechuga, tomate, cebolla morada y mayonesa

Cantidad:

-
1
+

[← Volver](#)

Back End

Marco Teórico

Esto complementa a la primera parte del proyecto. Para su desarrollo se hizo de las siguientes tecnologías:

- Java 21 JPA
- Hibernate
- Base de datos H2 en modo archivo.
- Lombok .

Java: lenguaje de programación y una plataforma informática que fue comercializada por primera vez en 1995 por Sun Microsystems.

Hibernate: es una herramienta de mapeo objeto-relacional (ORM) para la plataforma Java (y disponible también para .Net con el nombre de NHibernate) que facilita el mapeo de atributos entre una base de datos relacional tradicional y el modelo de objetos de una aplicación, mediante archivos declarativos (XML).

H2: es un sistema administrador de bases de datos relacionales programado en Java. Puede ser incorporado en aplicaciones Java o ejecutarse de modo cliente-servidor. Una de las características más importantes de H2 es que se puede integrar completamente en aplicaciones Java y acceder a la base de datos lanzando SQL directamente, sin tener que pasar por una conexión a través de sockets.

Lombok: Project Lombok es una biblioteca de Java que se integra automáticamente en tu editor y herramientas de construcción, dándole más sabor a tu Java. No vuelvas a escribir otro método getter o equals; con una sola anotación tu clase tendrá un builder completo, automatizará tus variables de registro (logging) y mucho más

Decisiones Técnicas y Arquitectura:

Para esta instancia se hizo uso de la plantilla sugerida por la, pero reutilizando código del segundo parcial.

La aplicación corre exclusivamente en modo consola, y no se comunica con el frontend:

Conceptos clave:

- Patrón Builder. Construcción fluida de entidades (Producto.builder()...build()), con @Builder.Default para inicializar colecciones y evitar NullPointerException.
- Mapeo Objeto-Relacional (ORM). Cada clase se mapea a una tabla; cada instancia, a una fila.
- Herencia con @MappedSuperclass. Base centraliza id, eliminado, createdAt y no genera tabla propia: sus columnas se replican en cada tabla hija. @PrePersist autocompleta createdAt.

- Generación de identidad. @GeneratedValue(strategy = GenerationType.IDENTITY): el ID lo asigna la BD en el INSERT.
- Relaciones unidireccionales. @OneToMany + @JoinColumn (la entidad dueña administra la FK) y @ManyToOne, con cascade = CascadeType.ALL y fetch = FetchType.LAZY según el caso.
- JPQL. Consultas orientadas a objetos (se navega c.productos, u.pedidos) con parámetros nombrados (:catId, :mail, :uid, :estado) y consultas tipadas que evitan casteos.
- Baja lógica (soft delete). No se borra: se marca eliminado = true y los listados filtran WHERE e.eliminado = false.
- Transaccionalidad ACID. begin()/commit()/rollback(). El alta de pedido es atómica: si algo falla, rollback() completo.
- Interfaz + polimorfismo. Calculable define calcularTotal(); Pedido la implementa sumando subtotales.
- Patrón Builder. Construcción fluida de entidades (Producto.builder()...build()), con @Builder.Default para inicializar colecciones y evitar NullPointerException.
- Un EntityManager por operación: cada método lo abre al inicio y lo cierra en finally. Las escrituras (guardar, eliminarLogico) usan transacción con rollback() en el catch por si llegan a fallar.

Manejo de errores:

- Validación de entrada con do-while que reintenta hasta recibir un valor válido (nombre no vacío, precio > 0, stock >= 0, opción de menú correcta).
- try/catch (NumberFormatException) en cada parseo de ID/número, informando sin cortar el programa.
- Optional<T> para expresar ausencia de resultados (buscarPorId, buscarPorMail) y forzar su verificación.
- rollback() en escrituras y en el alta de pedido.

Persistencia y configuración: H2 en modo archivo para que los datos sobrevivan entre ejecuciones sin servidor externo. JPAUtil mantiene un único EntityManagerFactory (singleton, porque su creación es costosa), cerrado con la opción 0 (JPAUtil.close()).

Problemas y soluciones

La plantilla ayudo debido a que ya tenía las anotaciones y las relaciones establecidas, así como también algunos métodos. Lo cual redujo drásticamente la posibilidad de errores, siendo en su mayoría simplemente de sintaxis y no merecen mención.

Sin embargo, en su momento me topé con el siguiente error, que luego pude resolver. No se si fue la mejor manera, pero quedo funcionando:

En alta de productos, un fragmento de la consigna indica “productoRepo.guardar”. El inconveniente es que los productos se administran desde Categorías y no desde productos. El producto se guarda en el repositorio de productos, pero no queda referencia en categorías.

Para resolverlo, se me ocurrió:

- I. Recuperar toda la lista de productos de la categoría,
- II. Crear un set con la lista de productos recibida anteriormente
- III. guardar el producto en el set
- IV. Recuperar la categoría entera del repositorio
- V. Cambiar el set de la categoría con el set nuevo, mediante el método .setProductos()
- VI. Guardar la categoría entera en el repositorio mediante el método .guardar()

```
List<Producto>productosExistentes=categoriaRepo.buscarProductosPor
Categoria(idCategoriaElegida);
Set<Producto>productos=new HashSet<>(productosExistentes);
productos.add(nuevoProducto);

Categoria categoria=cat.get();
categoria.setProductos(productos);
categoriaRepo.guardar(categoria);
```


Bibliografía

Wikipedia. (2026, 27 de junio). *HTML*. Wikipedia, la enciclopedia libre. Recuperado el 30 de junio de 2026, de <https://es.wikipedia.org/wiki/HTML>

Wikipedia. (2026, 17 de junio). *CSS*. Wikipedia, la enciclopedia libre. Recuperado el 30 de junio de 2026, de <https://es.wikipedia.org/wiki/CSS>

Vite. (s.f.). *Getting Started*. Vite. Recuperado el 30 de junio de 2026, de <https://vite.dev/guide/>

Microsoft. (s.f.). *TypeScript: JavaScript with syntax for types*. TypeScript. <https://www.typescriptlang.org>

Wikipedia. (2026, 1 de julio). *Java (lenguaje de programación)*. [https://es.wikipedia.org/wiki/Java_\(lenguaje_de_programaci%C3%B3n\)](https://es.wikipedia.org/wiki/Java_(lenguaje_de_programaci%C3%B3n))

Wikipedia. (2026, 1 de julio). *Hibernate*. <https://es.wikipedia.org/wiki/Hibernate>

Wikipedia. (2026, 1 de julio). *H2 (DBMS)*. [https://es.wikipedia.org/wiki/H2_\(DBMS\)](https://es.wikipedia.org/wiki/H2_(DBMS))